

APPLIES TO ALL 4 PHASE PROJECTS

Vibe Coding Helper Doc

Claude Code setup, prompt craft, and shipping to Vercel —
the workflow, not the theory.

Use alongside any Phase 1–4 Step-by-Step Guide

AI Engineering Masters Program

What This Doc Is

A practical workflow for building one of the four phase projects with an AI coding assistant (Claude Code, or an equivalent agentic coding tool) and deploying the result to Vercel. It's written once here because the workflow doesn't change between phases — only your prompts' content does.

1 · Setting Up Claude Code

- Install: `npm install -g @anthropic-ai/claude-code` (Node.js 18+ required).
- Run `claude` inside your project folder to start a session scoped to that repo.
- Authenticate with your Anthropic account when prompted — this happens once per machine.
- Sanity check: ask it to list the files in your repo. If it can see your folder structure, you're set up correctly.

If You're Using a Different Tool

Everything in this doc — ideation, prompt structure, iteration loop, Vercel deployment — applies equally to Cursor, Windsurf, or plain Claude in a chat window with copy-pasted code. Claude Code is the recommended default because it can read, edit, and run your project directly.

2 · Ideation Before You Write a Single Prompt

Don't open the coding tool first. Spend 15–20 minutes on paper or in a scratch doc answering: What's the one thing this project must prove works (per your phase's Problem Statement)? What's the smallest domain-specific dataset that makes it real, not a toy? What's explicitly out of scope so you don't scope-creep into the next phase's topics? Bring this into your first prompt — an assistant given a clear spec builds faster and straighter than one given a vague goal.

3 · Writing Prompts That Actually Work

Structure every build prompt with four parts

1. **Context** — what exists already (paste relevant file names/structure, not whole files if avoidable).
2. **Goal** — the single capability you want added this prompt, matching one step from your Step-by-Step Guide.
3. **Constraints** — tech stack, file boundaries ('don't touch the frontend'), style conventions already in use.
4. **Definition of done** — what you'll run to confirm it worked (a command, an endpoint, a test).

Context: FastAPI backend in backend/app/, no auth yet.
Goal: add PDF upload endpoint that chunks and embeds locally,
matching Step 3-4 of the Phase 1 guide.
Constraints: sentence-transformers only, no external embedding API.
Keep existing /health route untouched.
Done when: POST /api/upload with a test PDF returns a chunk count
and I can query FAISS for it from a python shell.

Rules that save you real time

- One capability per prompt. A prompt asking for retrieval + re-ranking + UI at once produces something you can't debug when it half-works.
- Ask for a plan before code on anything non-trivial: "outline the approach before writing files" catches wrong assumptions before they're 200 lines deep.
- Paste actual error output, not a paraphrase — exact stack traces let the assistant fix the real problem instead of guessing.
- After every meaningful change, ask it to update the README section for what changed — don't leave documentation for the end.

4 · The Iteration Loop

1. Prompt for one step from your phase's Step-by-Step Guide.
2. Run the checklist item for that step yourself — don't just trust the assistant's claim that it works.
3. If it fails, paste the exact error/output back in the same session (context matters — don't start a new chat mid-debug).
4. Once the checklist passes, commit with a message naming the step (e.g. "Step 4: FAISS index + query function").
5. Move to the next step. Resist the urge to let the assistant "also clean up" unrelated code mid-step — that's how sessions drift off scope.

5 · Connecting & Deploying to Vercel

A Install & Log In

```
npm install -g vercel  
vercel login
```

B Link Your Project

From your repo root: `vercel` — answer the prompts (link to existing project or create new, confirm root directory).
For a split frontend/backend repo, most teams deploy the frontend to Vercel and the backend to a separate host (Render, Railway, Fly.io) — decide this before your first deploy, not after.

C Environment Variables

- Never commit real API keys — `.env` must be gitignored.
- Add every variable from your `.env.example` in the Vercel dashboard (Project → Settings → Environment Variables), or via `vercel env add`.
- Set the frontend's API URL env var to your deployed backend's real URL, not localhost.

D Deploy & Verify

```
vercel --prod
```

After deploy, open the live URL in an incognito window (not your dev browser — avoid cached local state fooling you) and run through your phase's step-by-step checklist against the **deployed** app, not localhost. A project that only works locally is not a finished submission.

Common Deploy Failures

Blank page after deploy → almost always a missing/incorrect environment variable, check the Vercel function logs first. CORS errors → your backend's allowed-origins list doesn't include the Vercel domain yet. Backend timeout on Vercel → long-running agent/RAG calls may exceed serverless function limits; this is exactly why a separate backend host is often the right call for Phase 3/4 projects with multi-step agent runs.